

Chapter 8

Linux Isolation, Security 그리고 Final Project

5 Lecture · 영상 86 분 · Ubuntu Server 26.04 + C
Observe → Understand → Implement → Analyze

L36

Linux Capability

L37

seccomp

L38

Namespace

L39

Cgroup

L40

Final Project: Linux Event & Process
Monitor

L36

Linux Capability

Chapter 8. Linux Isolation, Security 그리고 Final Project
root 의 권한을 약 40 개로 쪼갬다

영상 16 분

▶ 애니메이션 [C8-1 Capability 검사](#)

이 Chapter 의 질문

	장치	질문
L36	Capability	어떤 특권을 가졌는가
L37	seccomp	어떤 System Call 을 할 수 있는가
L38	Namespace	무엇이 보이는가
L39	Cgroup	얼마나 쓸 수 있는가
L40	Final Project	전부를 하나의 프로그램으로

네 가지를 합치면 **Container**. Docker 는 새 Kernel 기능이 아니라 조합 도구

전부 아니면 전무

```
if (euid == 0) /* 모든 검사를 통과 */
```

```
-rwsr-xr-x 1 root root ... /usr/bin/passwd ← setuid root
```

필요한 것은 File 하나를 고칠 권한 . 주어지는 것은 **root** 의 전부

setuid root 프로그램의 Bug = 권한 상승 취약점

root 의 권한을 쪼갬다

		일반 사용자	root
/etc/shadow 읽기	CAP_DAC_READ_SEARCH	Permission denied	성공
nice(-5)	CAP_SYS_NICE	Permission denied	성공
chown	CAP_CHOWN	Not permitted	성공
mlock 256MB	CAP_IPC_LOCK	Cannot allocate	성공

Kernel 의 검사 = `capable(CAP_XXX)`. root = " 모든 Capability 를 가진 사용자 "

`CAP_SYS_PTRACE(L32) · CAP_PERFMON, CAP_BPF(L33, L35) · CAP_SYS_ADMIN`
= " 새로운 root "

→ Terminal 전환

다섯 집합

집합	의미
Effective	지금 검사에 쓰이는 것
Permitted	Effective 로 켤 수 있는 상한 . 버리면 다시 못 얻는다
Inheritable	exec 후 물려줄 수 있는 것
Bounding	자손까지 포함해 넘을 수 없는 천장
Ambient	일반 프로그램을 exec 해도 유지

/proc/PID/status 의 CapEff ... · capsh --decode= · getpcaps

→ Terminal 전환

권한 감사의 흐름

화면 전환 → [animation.html](#)

root → 일반 사용자 → File Capability → 권한 내려놓기

▶ 애니메이션 [C8-1 Capability 검사](#)

File Capability

```
sudo setcap cap_sys_nice+ep ./capdemo
./capdemo                                # uid 1000 인데 nice(-5) 만 성공
```

setuid root 였다면 System 전체 . 지금은 " 우선순위를 올릴 권한 " 뿐

```
/usr/bin/ping cap_net_raw=ep             ← 과거에는 setuid root
```

복사하면 사라지고 , 수정하면 **Kernel** 이 지운다 . 다시 Build 하면 다시 setcap

→ Terminal 전환

권한 내려놓기

```
prctl(PR_SET_KEEPCAPS, 1);          /* uid 를 바꿔도 Permitted 유지 */
setgroups(0, NULL); setresgid(...); setresuid(...); /* 순서 : Group → gid → uid */
caps = cap_init(); cap_set_flag(... keep ...); cap_set_proc(caps);
prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0);
```

1. root /etc/shadow: 읽힘 nice(-5): 성공
2. nobody + CAP_SYS_NICE /etc/shadow: denied nice(-5): 성공
3. root 로 돌아가기 : operation not permitted ← 되돌릴 수 없다

→ Code + Terminal

Bounding Set, NoNewPrivs, Container

Bounding Set: 자손이 setuid 나 File Capability 로도 얻을 수 없다

NO_NEW_PRIVS: 이후의 exec 가 권한을 늘리는 모든 경로를 무시 . 끝 수 없고 상속 .
seccomp 의 전제 조건

```
$ docker run --rm ubuntu grep Cap /proc/self/status  
CapEff: 00000000a80425fb          ← root 인데 14 개뿐
```

SYS_ADMIN, SYS_PTRACE, BPF, PERFMON 없음 → Chapter 7 의 도구가 안 되던 이유

정리

- `euid 0` = 전부 . `setuid root` 의 위험

Capability = root 를 40 개로 . `capable()` 은 **Effective** 를 본다

File Capability: `setcap cap_xxx+ep`

- 내려놓기 : `KEEPCAPS` → `Group/gid/uid` → `cap_set_proc` → `NO_NEW_PRIVS`
- Container 의 root = Capability 가 줄어든 root

다음 : L37. 특권이 필요 없는 System Call 은 ?

L37

seccomp

Process 가 호출할 수 있는 System Call 을 걸러낸다 . Filter 는 BPF Program

영상 16 분

▶ 애니메이션 [C8-2 seccomp Filter](#)

무엇을 막으려는가

System Call 약 400 개 = Kernel 로 들어가는 문 400 개 . 보통의 프로그램이 쓰는 것은 수십 개

장악당했을 때	Capability	seccomp
execve("/bin/sh")	못 막는다	막는다
socket/connect 로 유출	못 막는다	막는다
드문 System Call 의 Kernel 취약점	못 막는다	호출 자체가 불가

Strict Mode

```
prctl(PR_SET_SECCOMP, SECCOMP_MODE_STRICT);    /* read, write, _exit, sigreturn 만 */
```

1. Strict Mode 진입 후: write 는 허용된다
2. 이제 open 을 시도한다 ...
Killed

exit=137

→ *Terminal 전환*

Filter Mode: BPF Program

```
struct seccomp_data { int nr; __u32 arch; __u64 instruction_pointer; __u64 args[6]; };
```

```
BPF_STMT(LD, offsetof(seccomp_data, arch)); BPF_JUMP(JEQ, AUDIT_ARCH_X86_64, 1, 0); RET KILL;  
BPF_STMT(LD, offsetof(seccomp_data, nr)); BPF_JUMP(JEQ, SYS_mkdir, ...);  
RET ALLOW; RET ERRNO|EACCES;
```

mkdir → **Permission denied** ← Kernel 은 **mkdir** 을 실행조차 하지 않았다

Seccomp: 2 NoNewPrivs: 1

되돌릴 수 없다 · 상속된다 · NO_NEW_PRIVS 필수 · arch 먼저 검사

→ *Code + Terminal*

System Call 이 Filter 를 지나는 과정

화면 전환 → [animation.html](#)

ALLOW → ERRNO → KILL → io_uring 우회

▶ 애니메이션 [C8-2 seccomp Filter](#)

libseccomp 로 Allowlist

```
ctx = seccomp_init(SCMP_ACT_ERRNO(EPERM));           /* 기본 : 거부 */
seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(read), 0);
seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(write), 1, SCMP_A0(SCMP_CMP_LE, 2)); /* fd ≤ 2 만 */
seccomp_load(ctx);
```

- | | | |
|----------------------------|---------------------------|---------|
| 1. write(stdout) | → 허용 | |
| 2. write(fd 3, 이미 열린 File) | → Operation not permitted | ← 인자 검사 |
| 3. open("/etc/passwd") | → Operation not permitted | |
| 4. fork() | → Operation not permitted | |

Denylist 가 아니라 Allowlist. 새 System Call 은 계속 추가된다

→ Terminal 전환

위반하면

Action	동작	용도
ERRNO(e)	실행 않고 errno	호환성 (Docker 기본)
KILL_PROCESS	SIGSYS 로 종료	가장 안전
LOG	허용하되 기록	목록 작성 중
TRAP, NOTIFY	Handler / Supervisor	

```
audit: type=1326 ... comm="sandbox" sig=31 syscall=1 ...
```

← 어떤 호출 때문에 죽었는가

ERRNO 는 조용한 오동작을 만들 수 있다 → `strace -Z`

Allowlist 는 어떻게 만드는가

strace -c -f 로 수집 (오류 경로 , 종료 경로 포함)

1. 초기화와 본 작업을 나눈다 → Filter 는 초기화 뒤에 설치

SCMP_ACT_LOG 로 시험 운영

2. ERRNO / KILL 로 전환

glibc Upgrade 후에 죽는 것이 가장 흔한 운영 문제 (open→openat, clone→clone3, rseq)

한계 , 쓰이는 곳

포인터 인자를 볼 수 없다 : `openat` 의 경로 문자열은 검사 불가 → Landlock, AppArmor, SELinux

io_uring 은 Filter 를 지나지 않는다 (L27): SQE 의 `openat/read` 는 System Call 이 아니다
→ Docker, Android 가 `io_uring_*` 자체를 차단하는 이유

Docker / containerd	기본 Profile: <code>mount, bpf, perf_event_open, io_uring_*</code> ... 차단
systemd	<code>SystemCallFilter=@system-service</code>
Chrome, OpenSSH, Android	Sandbox

정리

- System Call 을 걸러 공격 표면을 줄인다 . Capability 와 상호 보완
Filter = seccomp_data 를 입력으로 받는 BPF. 되돌릴 수 없고 상속
libseccomp + **Allowlist** + 인자 조건
- ERRNO / KILL / LOG
- 초기화 뒤에 설치 . Library Upgrade 주의
- 경로는 못 본다 . io_uring 은 우회

다음 : L38. Process 에게 보이는 세계를 바꾼다

L38

Namespace

Process 에게 보이는 세계를 바꾼다 . Container 의 " 격리 " 는 이것

영상 17 분

▶ **애니메이션** [C8-3 Namespace](#)

종류

Namespace	격리하는 것
mnt	Mount Tree
uts	hostname
ipc	System V IPC, POSIX MQ
pid	Process 번호 . 첫 Process 가 PID 1
net	Interface, Routing, Port
user	uid/gid, Capability. 일반 사용자도 생성 가능
cgroup, time	Cgroup Root, Clock Offset

Kernel 은 하나 . VM 은 Kernel 을 여러 개 , Namespace 는 한 Kernel 이 Process 마다 다른 답을

내가 속한 Namespace

```
$ ls -l /proc/$$/ns  
mnt -> 'mnt:[4026531832]'    pid -> 'pid:[4026531836]'    uts -> 'uts:[4026531838]' ...
```

번호가 같으면 같은 Namespace · lsns

→ *Terminal* 전환

unshare 로 맛보기

```
$ sudo unshare --uts sh -c 'hostname lab-ns; hostname'      → lab-ns      ( 밖은 그대로 )  
$ sudo unshare --pid --fork --mount-proc sh -c 'echo $$; ps -e'
```

1

PID	TTY	CMD
1	?	sh
2	?	ps

--fork: PID Namespace 는 다음 **Child** 부터 적용

--mount-proc: ps 는 /proc 을 읽는다 → 새 /proc 을 Mount 해야 안쪽만 보인다

→ *Terminal 전환*

같은 Process, 다른 번호

화면 전환 → [animation.html](#)

PID Namespace → UTS / Mount → User Namespace

▶ 애니메이션 [C8-3 Namespace](#)

clone() 으로 만드는 가장 작은 Container

```
clone(child_main, stack_top, CLONE_NEWUTS | CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWIPC | SIGCHLD, argv);
sethostname("container", 9);
mount(NULL, "/", NULL, MS_REC | MS_PRIVATE, NULL);          /* Host 로 전파되지 않게 */
mount("proc", "/proc", "proc", ...);
execvp(...);
```

```
[host]      Child 의 pid = 860
[container] pid = 1, ppid = 0, hostname = container
```

PID 1: 고아를 입양 · Handler 없는 Signal 은 무시 · 죽으면 Namespace 전체가 SIGKILL

→ *Code + Terminal*

밖에서 보기 , 안으로 들어가기

```
PID self  mnt:[...1832] uts:[...1838] pid:[...1836] net:[...1833]
PID 869   mnt:[...2210] uts:[...2211] pid:[...2215] net:[...1833] ← Flag 를 준 것만 다르다
```

Host 에서는 Container 안이 그대로 보인다 . ps, kill, strace, perf 모두 통한다

nsenter -t PID -p -m ps -e = setns() = **docker exec**

clone	새 Namespace 에서 Child
unshare	자신을
setns	기존 Namespace 로

→ Terminal 전환

User Namespace: root 없이 root

```
$ ./usersns                                # sudo 없이
밖 : uid=1000
안 : uid=0
    sethostname: 성공                       ← 이 User NS 가 소유한 UTS NS 에 대해서는 CAP_SYS_ADMIN
    /etc/shadow: Permission denied         ← Host 자원에는 여전히 uid 1000
```

Rootless Container, Browser Sandbox, Flatpak

공격 표면이 넓어진다 → **Ubuntu 24.04+** 는 **AppArmor** 로 기본 제한

→ *Terminal 전환*

Namespace 가 격리하지 않는 것

Kernel 자체 : 취약점 하나가 전체의 탈출구 → seccomp, Capability 로 보완 . 더 강하게는 gVisor, Kata, Firecracker

자원 사용량 → Cgroup (L39)

/proc/meminfo, /proc/cpuinfo 는 Host 의 값

Container = Namespace + Cgroup + Capability + seccomp + Root File System

정리

- 8 종 . Kernel 은 하나
/proc/PID/ns, lsns
- clone / unshare / setns. unshare, nsenter 명령
- PID NS: 밖 860 = 안 1. /proc 재 Mount, MS_PRIVATE, PID 1 의 특성
- User NS: 안에서만 root
- Kernel 과 자원 사용량은 격리되지 않는다

다음 : L39. 얼마나 쓸 수 있는가

L39

Cgroup

Namespace = 무엇이 보이는가 . Cgroup = 얼마나 쓸 수 있는가

영상 17 분

▶ 애니메이션 [C8-4 Cgroup](#)

왜 필요한가

- Container 하나가 CPU 를 100% 쓴다
- Memory Leak 이 Host 전체를 OOM 으로 몰고 간다
- Fork Bomb
- " 이 Service 가 지난 한 시간 동안 CPU 를 얼마나 썼는가 "

`ulimit` = **Process** 하나의 한도 . `Cgroup` = 묶음의 합계 . 자손이 아무리 늘어도

Cgroup v2 의 구조

```
/sys/fs/cgroup/  
  cgroup.procs  
  cgroup.controllers  
  cgroup.subtree_control  
  system.slice/ssh.service/...
```

← **Directory = Group**. 모든 Process 는 정확히 한 Group 에
PID 를 쓰면 이동
쓸 수 있는 Controller: cpu memory pids io cpuset
Child 에게 커 줄 Controller

mkdir = Group 생성 . Kernel 이 제어용 File 을 채워 준다 (tracefs 와 같은 방식)

"No Internal Process": Process 는 Leaf 에만

→ *Terminal 전환*

CPU 한도 : `cpu.max`

```
cpu.max = "20000 100000"      100ms 주기마다 20ms 까지만
[ 1 s] 받은 CPU 시간 0.22 s → 20.7 %
합계 : 벽시계 5.06 s 동안 CPU 1.02 s (20.1 %)
nr_periods 51      nr_throttled 50      throttled_usec 4012790
```


 ■ 실행 ▒ Throttled

→ *Terminal 전환*

Throttling 은 Latency 의 숨은 원인

평균 사용률이 한도보다 낮아도 , **몰아서 쓰면** 남은 주기 동안 Container 전체가 멈춘다

- Thread 가 많을수록 quota 를 빨리 소진 (quota 는 Group 의 합)

프로그램 안에는 **아무 흔적이 없다** . 진단 : `cpu.stat` 의 **`nr_throttled`**

	<code>cpu.max</code>	<code>cpu.weight</code>
의미	절대 상한	경쟁할 때의 비율
한가할 때	그래도 제한	다 쓴다
Kubernetes	<code>limits.cpu</code>	<code>requests.cpu</code>

한도를 움직여 보기

화면 전환 → [animation.html](#)

Scenario 3 개 + Sandbox: CPU Limit / Memory Limit Slider

▶ 애니메이션 [C8-4 Cgroup](#)

Memory 한도 : `memory.max`

```
60 MB 사용 중  
Killed                               exit code = 137  
oom 1      oom_kill 1  
Memory cgroup out of memory: Killed process 975 (memhog) ... anon-rss:65280kB
```

한도에 닿으면 : ① Group 안에서 Reclaim → ② 안 되면 **OOM Kill (SIGKILL)**

malloc 은 실패하지 않는다 . 쓸 때 죽는다

Page Cache 도 계산된다 . 위험한 것은 `memory.stat` 의 **anon**

`memory.high`: 죽이지 않고 늦춘다 · **memory.events** 의 **oom_kill** 을 감시

→ *Terminal 전환*

Process 수 한도 : pids.max

```
fork #10 실패 : Resource temporarily unavailable ← pids.max  
만들어진 Child: 9 개
```

Fork Bomb 방어 . **Thread** 도 센다 → "unable to create native thread"

→ Terminal 전환

systemd 와 Container

```
system.slice/ssh.service      MemoryCurrent=5234688      CPUUsageNSec=1820394000
```

모든 **Service** 가 자기 **Cgroup** → `systemd-cgtop`, `systemd-run --scope -p CPUQuota=20% -p MemoryMax=64M`

<code>docker run --cpus=0.5</code>	<code>cpu.max = "50000 100000"</code>
<code>docker run -m 256m</code>	<code>memory.max</code>
<code>--pids-limit</code>	<code>pids.max</code>

Container 안에서 자기 한도를 알려면 `/proc/cpuinfo` 가 아니라 **`/sys/fs/cgroup/cpu.max`**

정리

Process 묶음의 한도와 집계

- v2: 단일 Tree, mkdir, cgroup.procs, subtree_control

cpu.max: Throttling. **nr_throttled**

memory.max: Reclaim → **OOM Kill (137)**. malloc 은 실패하지 않는다 . **oom_kill**

pids.max: fork EAGAIN

- systemd, Docker, Kubernetes = 이 File 들에 쓰기

다음 : L40. Final Project

L40

Final Project: Linux Event & Process Monitor

Event, Thread, Memory, System Call, Kernel, Tracing 을 하나의 프로그램에서 연결한다

영상 20 분

▶ 애니메이션 [C8-5 Final Project: Linux Event & Process Monitor](#)

무엇을 만드는가

요구 사항	사용하는 것	
주기적 Sample	timerfd	L25
CPU%, RSS, Thread, fd	/proc	L31
전체 Scan 은 Loop 밖에서	Worker + eventfd × 2	L25, L26
설정 Hot Reload	inotify	L25, L15
깨끗한 종료	signalfd	L25
빠르고 확실한 기록	mmap Ring Log + msync	L10, L15, L28
한 Thread, Lock 없이	epoll	L23
장악당해도 피해 최소	Capability + seccomp	L36, L37
자원 제한	Cgroup	L39
정말 그런가 ?	strace, perf, ftrace, bpftrace	L32~L35

구조

```
timerfd ┌
signalfd ┤ epoll_wait → switch (data.u32)
eventfd ┤
inotify ┤
        │   TIMER  : /proc 읽기 → 화면 + mlog_put() → monitor.log.bin (mmap)
        │           3 Tick 마다 write(req_efd) → Worker "scanner"
        │   WORKER : top 결과 출력 ← write(done_efd) — /proc 전체 Scan, mutex 로 결과 전달
        │   INOTIFY: monitor.conf 다시 읽기
        │   SIGNAL  : TERM → running = 0 → 정리
```

monitor.c · procstat.h · mlog.h · harden.h · logdump.c · load.c

실행

```
[sample ] load      pid 1029  R cpu  40.0% rss  1760 kB thr 1 fds 3
[scan   ] Process 52 개 중 CPU 상위:  load(1029) 40%
[stats  ] tick 4 (놓친 Tick 0), scan 1, reload 0, log 5 건      ← kill -USR1
[config ] interval = 500 ms, 대상 2 개                          ← mv monitor.conf.tmp monitor.conf
[signal ] 15 수신 (보낸 pid 1027) → Graceful Shutdown
[stop   ] 정상 종료
```

→ Terminal 전환 (Terminal 두 개)

Event Loop: 순서가 설계다

```
pthread_sigmask(SIG_BLOCK, ...);          /* 1. Thread 생성 전에 */
epoll_create1 ... timerfd ... signalfd ... eventfd × 2 ... inotify ... mlog_open /* 2. 권한이 있는 동안 전부 연다 */
pthread_create(worker); load_config();
if (hardened) { drop_privileges(); install_seccomp(); }          /* 3. 버린다 */
while (running) { n = epoll_wait(...); switch (...) { ... } }   /* 4. 기다리는 곳은 하나 */
/* 5. 정리 */
```

공유 상태에 **Lock** 이 없다 (Loop Thread 만 접근)

`expirations > 1 → n_missed++`: **Loop** 가 밀렸다는 증거를 스스로 기록

CPU% 는 실제 경과 시간으로 나눈다 · `ps_read` 실패는 정상 상황

→ *Code* 화면

Worker 와 두 개의 eventfd

	방향	Mode
req_efd	main → Worker	Blocking (할 일 없으면 잔다)
done_efd	Worker → main	Non-blocking + epoll

```

while (read(req_efd, ...) && !stopping) {
    scan_all(a); nanosleep(1s); scan_all(b);          /* 오래 걸리는 일 */
    lock; memcpy(top, best); unlock;                  /* 결과 복사에만 mutex */
    write(done_efd, &one, 8);
}

```

요청이 겹치면 Counter 에 **합산** · 종료 : stopping + write(req_efd) 로 깨워서 join

mmap Ring Log

```
ftruncate(fd, bytes);  hdr = mmap(..., MAP_SHARED, fd, 0);
m->rec[m->hdr->total % MLOG_CAP] = *r;          /* 내용을 먼저 */
m->hdr->total++;                                   /* 그 다음에 공개 (L28) */
```

기록에 System Call ?	없다
kill -9	남는다 (Dirty Page 는 Kernel 에)
전원 차단	최근 ~30 초 . 그래서 정상 종료 때만 msync(MS_SYNC)
logdump	실행 중에도 읽힌다 . 같은 Page Cache

→ Terminal 전환: `./logdump 4`

자기 격리 : -H

```
[harden ] uid=65534 로 전환 , Capability 는 CAP_SYS_PTRACE 와 CAP_DAC_READ_SEARCH 만 유지  
[harden ] seccomp Filter 설치 완료  
[sample ] systemd pid 1 ... fds 21 ← nobody 인데 읽힌다  
Uid: 65534   CapEff: 00000000000080004   NoNewPrivs: 1   Seccomp: 2
```

```
$ sudo kill -USR2 ...  
[attack ] system("id") 실패 (rc=32512) ← seccomp 가 차단 ( -H 없이는 uid=0(root) 가 출력된다 )
```

① 자원을 연다 → ② Thread → ③ uid/Capability → ④ seccomp (30 개 , 초기화 뒤라서 짧다)

→ Terminal 전환

전체 그림

화면 전환 → [animation.html](#)

Tick → Worker 완료 → 설정 변경 → SIGTERM → -H 의 시작 순서

▶ 애니메이션 [C8-5 Final Project: Linux Event & Process Monitor](#)

분석 : 설계대로 동작하는가

```
strace -c -f (6 초):  epoll_wait 7   openat 122 (Worker 의 /proc Scan)   write 14 ( 화면 )   msync 1
                    → Log 기록에 의한 write = 0
perf stat:          2.72 msec task-clock  # 0.000 CPUs utilized      context-switches 16
/proc/PID/task:     monitor ep_poll          scanner do_wait_intr_irq
/proc/PID/fd:       [eventpoll] [timerfd] [signalfd] [eventfd] [eventfd] inotify monitor.log.bin
/proc/PID/maps:     ... rw-s ...  monitor.log.bin
```

fttrace: Timer 만료 → sched_switch → epoll_wait return → openat 의 간격 = **반응 지연**

bpftrace: 실행 중의 System Call 집계 = **Allowlist 검증**

→ *Terminal 전환*

Cgroup 안에서 , 그리고 확장

```
sudo systemd-run --scope -p CPUQuota=5% -p MemoryMax=32M -p TasksMax=8 ./monitor -H ...
```

계층	장치
특권	Capability 2 개
System Call	seccomp 30 개
자원	Cgroup

확장 과제 : 이름으로 대상 찾기 · Cgroup 의 throttled/oom_kill 감지 · pidfd_open · io_uring 으로 /proc Scan(이득이 있는가 ?) · Log 방식 세 가지의 비용 비교

과정을 마치며

execve → ELF → ld.so → main

Chapter 1

mmap, Page Fault

Chapter 2

Page Cache, Dirty, Writeback, msync

Chapter 3

mutex(futex), Lock 없는 설계

Chapter 4

epoll + timerfd / signalfd / eventfd / inotify

Chapter 5

(File I/O가 병목이면 io_uring)

Chapter 6

/proc, strace, perf, ftrace, bpftrace

Chapter 7

Capability, seccomp, Namespace, Cgroup

Chapter 8

Observe → Understand → Implement → Analyze

과정을 마치며

Observe → Understand → Implement → Analyze

개념 워크시트

[worksheet.html](#)

Lecture 당 10 문항 · 50 문항

애니메이션

[animation.html](#)

5 종

강의 자료

[lecture.html](#)

빈칸 채우기 · 괄호 넣기

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog